

My Environment: Linux

Thread Numbers	Speed (tok/s)	User Time	System Time	Use Time/System Time
0	17.6621	14.9092	0.0420	354.980952381
1	15.0057	16.8432	0.4041	41.680772086
2	25.2190	17.8747	0.4005	44.630961298
4	30.5789	25.4882	0.5988	42.565464262
6	41.6534	28.9766	0.8880	32.631306306
8	52.1289	29.9114	1.0958	27.296404453
10	62.6944	30.5807	1.3527	22.607156058
12	26.8663	61.0552	3.9187	15.580473116
16	49.4993	31.4787	2.6297	11.970452903

seq.c vs 1 thread on parallel.c:

1 is even slower. System time larger because of the context switch between the main thread and the single thread created. User Time is also larger because it has more unnecessary codes to run.

thread 0 to 16 “use time/system time”:

This ratio goes down significantly from 0 to 1 because when I have more than 1 thread to manage in the process, more time is spent in system (kernel). As the thread increases, more time is spent on the context switching and the multi-core thread management (such as cache consistency).

Thread 1 to 10 “performance”:

The performance increases as the threads increase.

6 threads ->

```
length: 262, speed (tok/s): 42.6780
Thread 0 has completed - user: 3.7607 s, system: 0.0874 s
Thread 1 has completed - user: 4.3811 s, system: 0.0916 s
Thread 2 has completed - user: 5.1281 s, system: 0.0933 s
Thread 3 has completed - user: 4.9431 s, system: 0.1046 s
Thread 4 has completed - user: 4.7945 s, system: 0.0880 s
Thread 5 has completed - user: 4.5804 s, system: 0.0693 s
main thread - user: 0.5234 s, system: 0.3438 s
Whole process - user: 28.1395 s, system: 0.8503 s
```

10 threads ->

```

length: 262, speed (tok/s): 62.9354
Thread 0 has completed - user: 3.2237 s, system: 0.1243 s
Thread 1 has completed - user: 3.0734 s, system: 0.0966 s
Thread 2 has completed - user: 3.0988 s, system: 0.0996 s
Thread 3 has completed - user: 3.0632 s, system: 0.1029 s
Thread 4 has completed - user: 3.1405 s, system: 0.0800 s
Thread 5 has completed - user: 3.1155 s, system: 0.0837 s
Thread 6 has completed - user: 3.0902 s, system: 0.0959 s
Thread 7 has completed - user: 3.0542 s, system: 0.0931 s
Thread 8 has completed - user: 2.7911 s, system: 0.0978 s
Thread 9 has completed - user: 2.4092 s, system: 0.0843 s
main thread - user: 0.5310 s, system: 0.4109 s
Whole process - user: 30.5995 s, system: 1.3613 s

```

The user time of each thread decreased in 10 threads because each is responsible for less computation. However, the decrease is not as much as decreasing the total user time. The total user time actually increased. However, it is still much faster in the wall-clock time compared to 6 threads because these LWP are distributed across multi-cores. My laptop has 16 virtual cores.

Thread 10, 12, 16 performance:

This part is difficult to explain.

The performance is 10 >> 16 >> 12.

I used the following to find out the cores each thread was run at the end of the process.

```

int cpu = sched_getcpu();
printf("Thread %ld running on CPU core %d\n", (long)pthread_self(), cpu);

```

10 threads - Only core 3 was used twice. Utilizing each core without overloading their hyperthreads.

12 threads - Cores 4,6,7 were each used twice, probably oversubscribing it.

16 threads - A few cores were used more than 3 times, also oversubscribing.

Thus, we can see 10 threads are run very evenly among the CPUs without contending for them.

However, the above cannot explain thread 12-16 behaviors. Actually, the performance got better from thread 14.

I propose it's about the cache hierarchy and the bus by also taking into account that all the threads write to the same array (though on different part of it).

From 10 to 12 threads, there'll be more cache line sharing/invalidation especially on L3 cache which is shared by all the cores. Higher memory bus contention.

However, as I ran

```

perf stat -e cache-misses,cache-references ./parallel 10 42 "Fibonacci"

```

to find out the cache-misses, etc. the thread 10-16 performed similarly.

```
Thread 13 has completed - user: 1.0009 s, system: 0.1124 s
main thread - user: 0.7327 s, system: 1.2177 s
Whole process - user: 31.5553 s, system: 3.9149 s

Performance counter stats for './parallel 16 42 Fibonacci':

    558,304,015      cpu_atom/cache-misses/      #    41.86% of all cache refs      (72.80%)
    429,444,416      cpu_core/cache-misses/      #    32.20% of all cache refs      (27.25%)
    1,333,736,111      cpu_atom/cache-references/    (72.80%)
    1,605,237,014      cpu_core/cache-references/    (27.25%)

    5.574765321 seconds time elapsed

    31.555261000 seconds user
    3.921402000 seconds sys

Performance counter stats for './parallel 10 42 Fibonacci':

    567,151,627      cpu_atom/cache-misses/      #    47.51% of all cache refs      (78.64%)
    438,733,697      cpu_core/cache-misses/      #    36.75% of all cache refs      (21.42%)
    1,193,820,258      cpu_atom/cache-references/    (78.64%)
    1,359,607,472      cpu_core/cache-references/    (21.42%)

    4.440986794 seconds time elapsed

    30.962607000 seconds user
    2.164929000 seconds sys
```

Therefore, I think the only possible explanation of why performance drops from 11 to 13 and go back up from 13 to 16 is because of the specific work done in this program: matrix calculation and the multi-headed attention. It might be the constant being set to work better with 10, 11, 13-16 threads on my laptop Latitude 5440 Core i5.